```
#Krishna Kumar Trivedi
#Exp-6
# BLOCK 1: Import Libraries


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import StandardScaler
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
# BLOCK 2: Load Dataset

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

print("First 5 rows of dataset:\n")
print(df.head())
```

```
First 5 rows of dataset:

   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   species
0        0
1        0
2        0
3        0
4        0
```

```
# BLOCK 3: Define Features and Target

X = df.iloc[:, :-1]   # All columns except last
y = df.iloc[:, -1]    # Last column

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

```
Shape of X: (150, 4)
Shape of y: (150,)
```

```
# BLOCK 4: Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)
```

```
Training data size: (105, 4)
Testing data size: (45, 4)
```

```
# BLOCK 5: Train Model

model = GaussianNB()
model.fit(X_train, y_train)

print("Model trained successfully!")
```

```
Model trained successfully!
```

```
# BLOCK 6: Prediction

y_pred = model.predict(X_test)
```

```
print("First 10 Predictions:", y_pred[:10])
```

```
First 10 Predictions: [1 0 2 1 1 0 1 2 1 1]
```

```
# BLOCK 7: Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:\n")
print(cm)
```

```
Confusion Matrix:

[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```

```
# BLOCK 8: Calculate TP, FP, FN, TN

TP = np.diag(cm)
FP = np.sum(cm, axis=0) - TP
FN = np.sum(cm, axis=1) - TP
TN = np.sum(cm) - (TP + FP + FN)

print("True Positives (TP):", TP)
print("False Positives (FP):", FP)
print("False Negatives (FN):", FN)
print("True Negatives (TN):", TN)
```

```
True Positives (TP): [19 12 13]
False Positives (FP): [0 0 1]
False Negatives (FN): [0 1 0]
True Negatives (TN): [26 32 31]
```

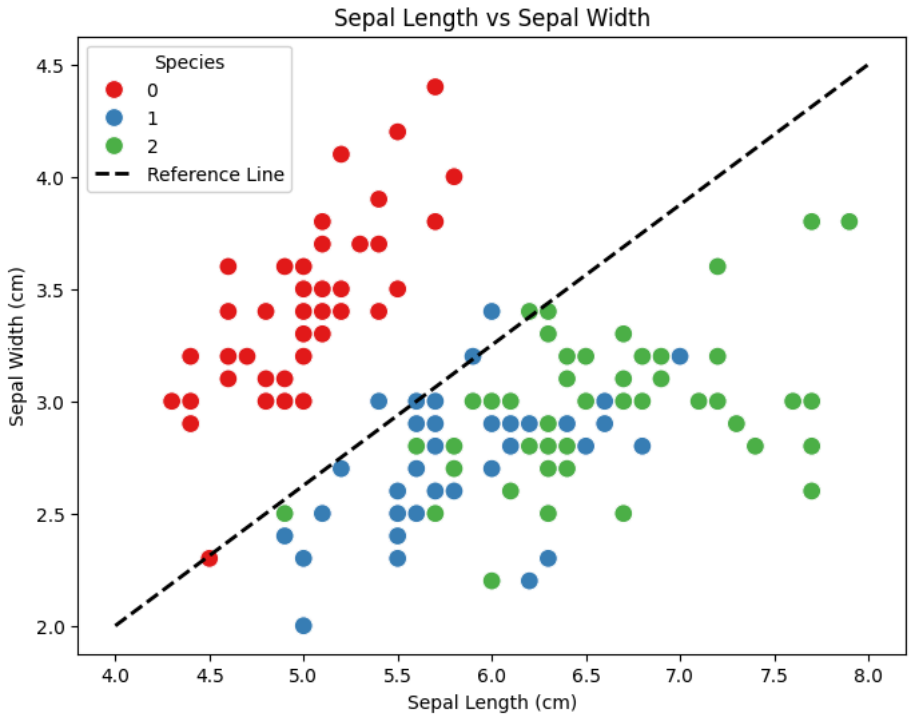
```
# BLOCK 9: Accuracy and Error Rate

accuracy = np.sum(TP) / np.sum(cm)
error_rate = 1 - accuracy

print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
```

```
Accuracy: 0.9777777777777777
Error Rate: 0.022222222222222254
```

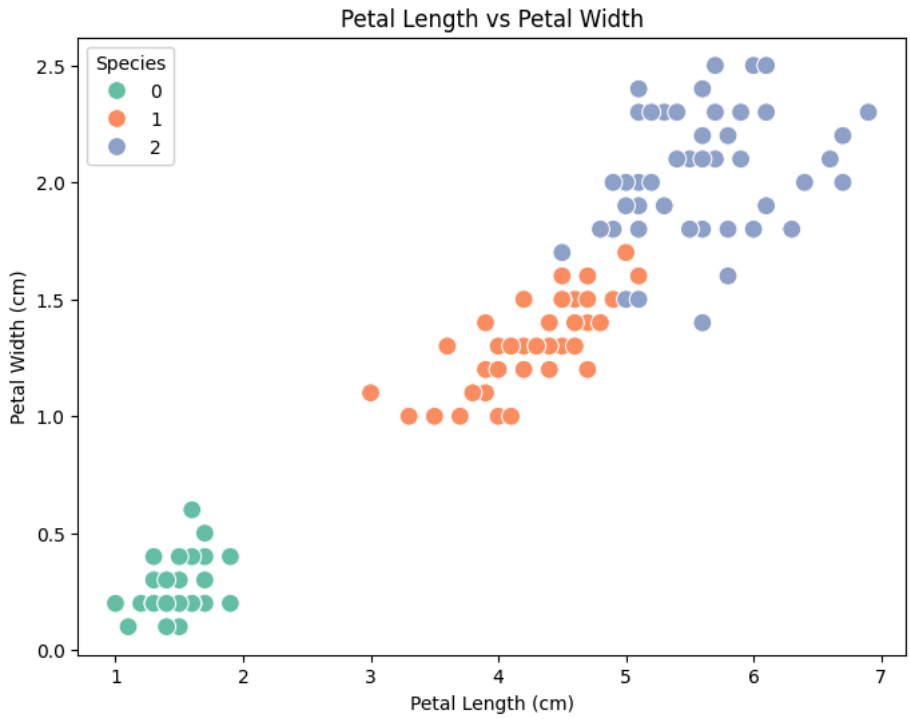
```
# BLOCK 10: Precision and Recall

precision = TP / (TP + FP)
recall = TP / (TP + FN)

print("Precision:", precision)
print("Recall:", recall)
```

```
Precision: [1.         1.         0.92857143]
Recall: [1.         0.92307692 1.        ]
```

```
plt.figure(figsize=(8, 6))
sns.scatterplot(
x=df['sepal length (cm)'],
y=df['sepal width (cm)'],
hue=df['species'],
palette='Set1',
s=100
)
plt.plot([4, 8], [2, 4.5], color='black', linestyle='--', linewidth=2, label='Reference Line')
plt.title("Sepal Length vs Sepal Width")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.legend(title='Species')
plt.show()
```




```
plt.figure(figsize=(8, 6))
sns.scatterplot(
x=df['petal length (cm)'],
y=df['petal width (cm)'],
hue=df['species'],
palette='Set2',
s=100
)
plt.title("Petal Length vs Petal Width")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.legend(title='Species')
plt.show()
```




Double-click (or enter) to edit